

Project Write-up

IITB Insti-Assist

1. Chosen Scope and Why

The objective of this project was to develop a Retrieval-Augmented Generation (RAG) assistant capable of answering questions about IIT Bombay using only a collection of provided documents. The assistant is intended to function as a factual information retrieval system rather than a general-purpose chatbot.

The project scope focuses on answering queries related to IIT Bombay policies, regulations, and informational documents such as hostel manuals, academic regulations, and Learners' Space resources. Instead of relying on the language model's internal knowledge, the assistant retrieves relevant information from indexed documents before generating a response. This approach minimizes hallucinations and ensures that responses remain grounded in the available data.

A strict system prompt instructs the language model to answer only using the retrieved information. If sufficient information is unavailable, the assistant returns the predefined response:

"I don't know based on the available documents."

This design improves reliability and makes the assistant suitable for document-based question answering.

2. Data Sources Used

The assistant supports the following document formats:

- PDF (.pdf)
- Plain text (.txt)
- Markdown (.md)

PDF files are processed using the **PyPDF** library, which extracts text page by page while preserving page numbers for citation. Text and Markdown files are read directly as plain text.

For every document, the following metadata is stored:

- Source filename
- Page number (for PDFs)

This metadata enables the assistant to display the source document and page used to generate each answer.

3. Chunking Strategy and Why

After loading the documents, unnecessary whitespace is removed before the text is divided into smaller chunks.

The implementation uses the following configuration:

- Chunk size: 800 characters
- Chunk overlap: 150 characters

Using overlapping chunks preserves context between adjacent chunks. Without overlap, important information located near chunk boundaries may be split across different chunks, reducing retrieval accuracy. The 150-character overlap ensures that related information is likely to appear in consecutive chunks, improving the quality of retrieved context.

Each chunk is converted into an embedding using the **all-MiniLM-L6-v2** SentenceTransformer model. The embeddings are normalized and indexed using **FAISS**, enabling efficient semantic similarity search.

When a user submits a query, it is embedded using the same model. FAISS retrieves the four most similar chunks, after which chunks with similarity scores below a predefined threshold are discarded. The remaining chunks form the context provided to the Gemini model for answer generation.

This retrieval-first approach improves response relevance while keeping the prompt concise.

4. Known Limitations and Future Improvements

Although the system performs well for document-grounded question answering, it has several limitations.

First, the assistant can answer only questions whose information exists within the indexed documents. If relevant information is unavailable, the system refuses to answer rather than generating unsupported content.

Second, PDF extraction relies on text-based PDFs. Scanned documents containing only images are not supported because Optical Character Recognition (OCR) has not been implemented.

The current implementation uses fixed-size character-based chunking. While simple and efficient, sentence-aware or semantic chunking techniques could preserve context more effectively and further improve retrieval performance.

The retrieval component currently relies only on FAISS similarity search. Performance could be enhanced by incorporating reranking models that reorder retrieved chunks according to their relevance before they are supplied to the language model.

Additional improvements with more development time include support for more document formats, conversation history for follow-up questions, highlighted citations within responses, and automatic updating of the vector index when documents are added or modified.

Overall, the project demonstrates a complete Retrieval-Augmented Generation pipeline consisting of document ingestion, chunking, embedding generation, semantic retrieval, and grounded answer generation while ensuring that responses remain faithful to the provided IIT Bombay documentation.